Lớp: **IT003.R17**

BÁO CÁO KẾT QUẢ THỬ NGHIỆM

Thời gian thực hiện: 01/10 – 10/10/2026

Sinh viên thực hiện: Hoàng Minh Quân - 25521486

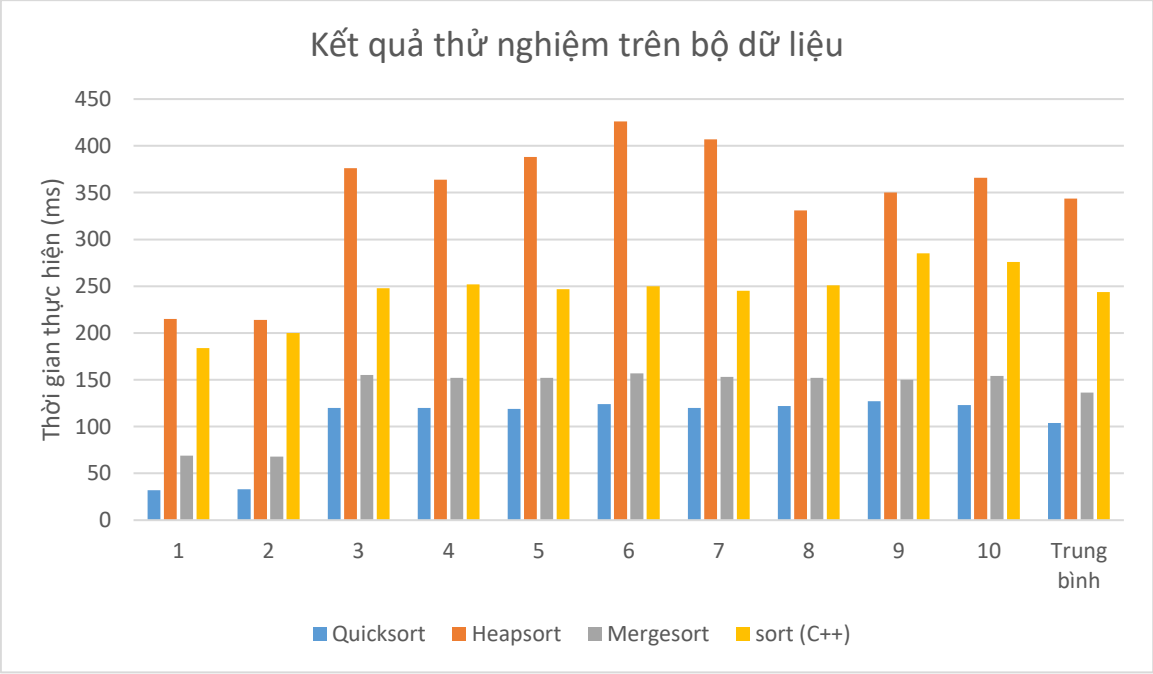
Nội dung báo cáo:

I. Kết quả thử nghiệm

1. Bảng thời gian thực hiện

Dữ liệu	Thời gian thực hiện (ms)			
	Quicksort	Heapsort	Mergesort	sort (C++)
1	32	215	69	184
2	33	214	68	200
3	120	376	155	248
4	120	364	152	252
5	119	388	152	247
6	124	426	157	250
7	120	407	153	245
8	122	331	152	251
9	127	350	150	285
10	123	366	154	276
Trung bình	104	343.7	136.2	243.8

2. Biểu đồ (cột) thời gian thực hiện



II. Kết luận:

Qua kết quả thử nghiệm trên 10 bộ dữ liệu, có thể thấy thời gian thực hiện của các thuật toán sắp xếp có sự khác biệt đáng kể.

1. QuickSort

- Ưu điểm: Có thời gian thực hiện trung bình thấp nhất trong thử nghiệm, đạt 104 ms. Thuật toán thường có hiệu năng tốt trên các bộ dữ liệu lớn và không cần nhiều bộ nhớ phụ.
- Nhược điểm: Hiệu năng phụ thuộc vào cách chọn phần tử chốt (pivot). Nếu chọn pivot không phù hợp, thuật toán có thể đạt độ phức tạp $O(n^2)$.

2. HeapSort

- Ưu điểm: Có độ phức tạp thời gian $O(n \log n)$ trong mọi trường hợp. Thuật toán không cần mảng phụ lớn và hiệu năng không phụ thuộc nhiều vào thứ tự ban đầu của dữ liệu.
- Nhược điểm: Thuật toán không đảm bảo thứ tự bằng nhau của dữ liệu sau khi sort, thường có khả năng tận dụng bộ nhớ đệm kém hơn QuickSort và MergeSort nên tốc độ thấp hơn dù độ phức tạp tốt.

3. MergeSort

- Ưu điểm: Có độ phức tạp thời gian $O(n \log n)$ trong mọi trường hợp Thời gian thực hiện tương đối ổn định giữa các bộ dữ liệu, đồng thời thuật toán giữ nguyên thứ tự của các phần tử có giá trị bằng nhau sau khi sort.
- Nhược điểm: Cần thêm bộ nhớ phụ để trộn các dãy con. Với mảng lớn, lượng bộ nhớ bổ sung này có thể đáng kể.

4. Hàm sort() trong C++

- Ưu điểm: Dễ sử dụng, ngắn gọn và được tối ưu trong thư viện chuẩn C++. Hàm có độ phức tạp thời gian $O(n \log n)$trong trường hợp xấu theo tiêu chuẩn C++ hiện hành.
- Nhược điểm: Người dùng ít kiểm soát được cách triển khai bên trong nếu chỉ sử dụng trực tiếp hàm thư viện. Hàm std::sort() cũng không bảo đảm tính ổn định đối với các phần tử có giá trị bằng nhau.

III. Thông tin chi tiết – link github, trong repo gibub cần có

1. Báo cáo
2. Mã nguồn
3. Dữ liệu thử nghiệm

https://github.com/MQuan-fake/Thuc-nghiem-cac-thuat-toan-sap-xep